AgentCgroup

AI Agent Resource Characterization and OS-Level Control

Yusheng Zheng (yzhen165@ucsc.edu)
PhD Student

Roadmap

- Motivation & Problem
- Resource Characterization (144 tasks, 2 models)
- Agent vs. Cloud Workloads & Three Mismatches
- AgentCgroup: eBPF + Bash Wrapper + Negotiation
- Evaluation & Takeaways

Motivation

AI coding agents such as **Claude Code** and **Codex** are being deployed at scale, autonomously running compilers, test suites, and package managers in sandboxed containers.

New Workloads: AI Agents

- LLM reasons → calls tools → observes → iterates
- Agent **autonomously decides** what to run and when
- Perform actions 24/7; tool usage may be heavy and bursty

...How much does it actually cost?

- Providers already host **many concurrent agents** on shared infrastructure
- Are traditional **containers** and **VMs** enough? Do we need new tools?
- Most studies focus on safety, but what about **resource usage** and

Characterization: Setup

144 SWE-rebench tasks, using Claude Code with two LLM backends

Backend	Execution	Tasks
GLM-4.7-Flash	Local GPU	111
Claude Haiku 4.5	Cloud API	33

Hardware: Intel Core Ultra 9 285K, 24 cores, 128 GB DDR5

Environment: Podman containers, Linux 6.15, 1s sampling

Cross-model comparison: 33 overlapping tasks

What We Measure

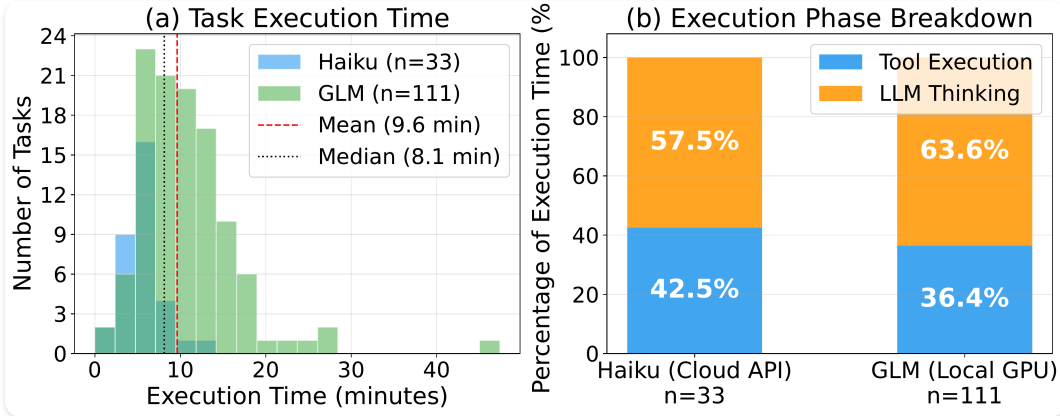
- **What does the agent do?** (execution model)
- **What does resource usage look like?** (resource dynamics)
- **Can demands be predicted?** (cross-task variance)

Container Images

- **Average: 3.5 GB, 7× larger than microservices and 70× larger than FaaS**

Execution Model

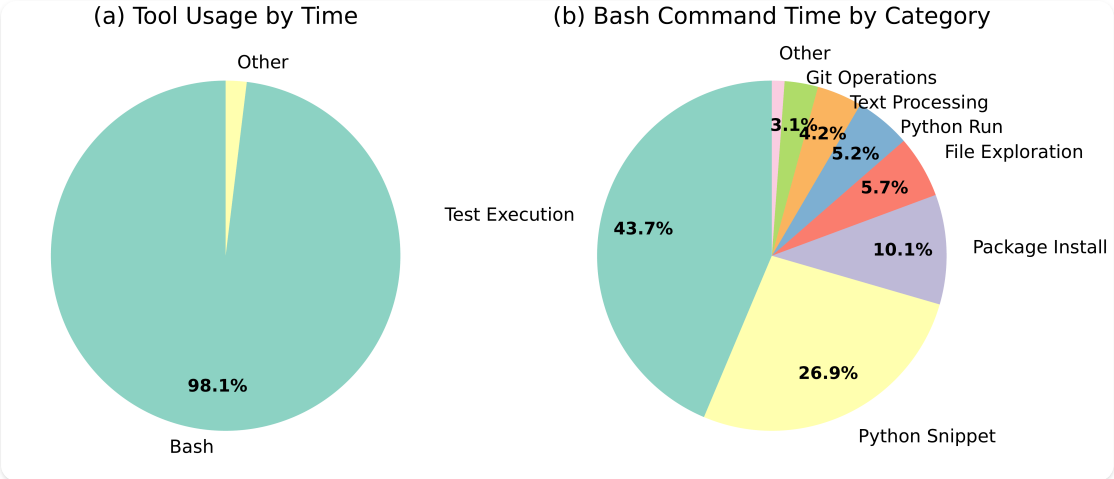
Q: Where does the time go?



- Task duration: mean 9.6 min
- LLM reasoning: 26–44% of end-to-end task latency
- The remainder:
  - tool execution (~40% of active time)
  - setup env and initialization (29–45%)

What Tool Dominates?

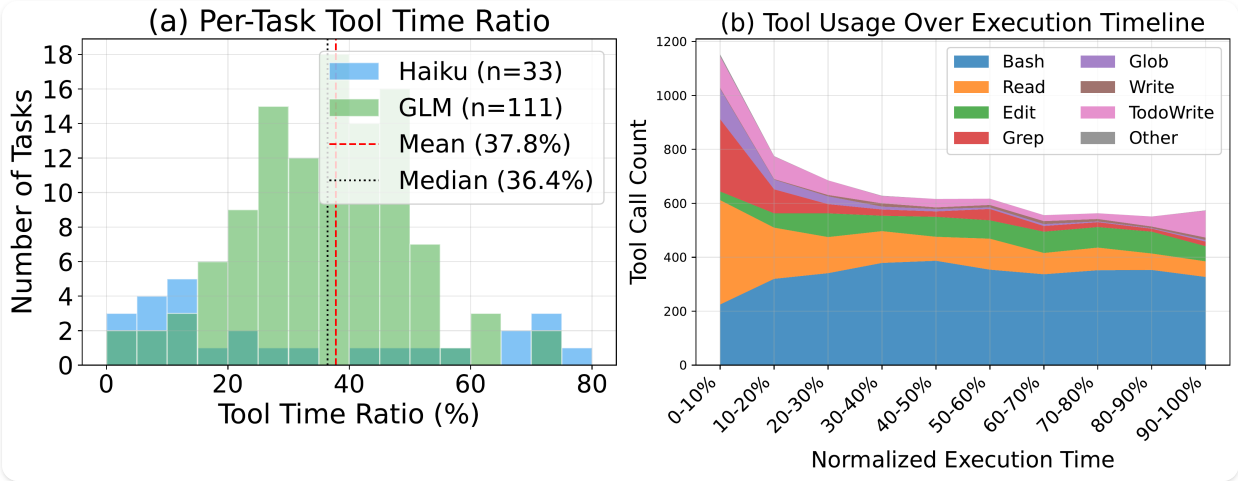
Q: Which tool types consume the most execution time?



- Bash dominates tool execution
- Top bash categories: test execution, Python snippet, package install

Tool Execution: Temporal Pattern

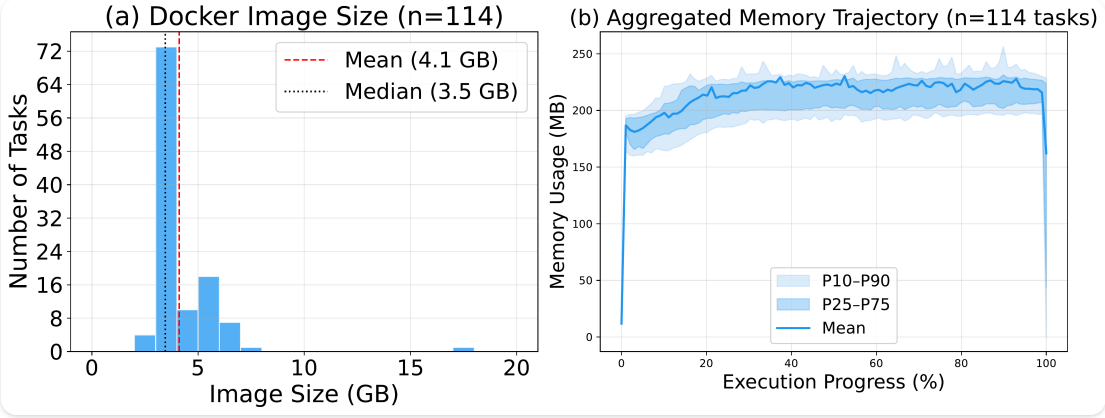
Q: When do different tools execute during a task?



- Tool calls follow an "**understand, modify, verify**" pattern: reads dominate the first 30%, and Bash peaks at 40–80%

Memory & Resource Structure

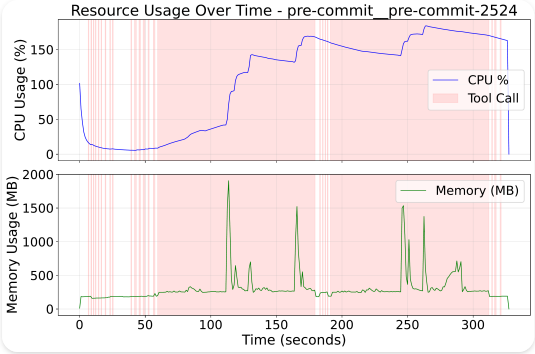
Q: What does resource consumption look like?

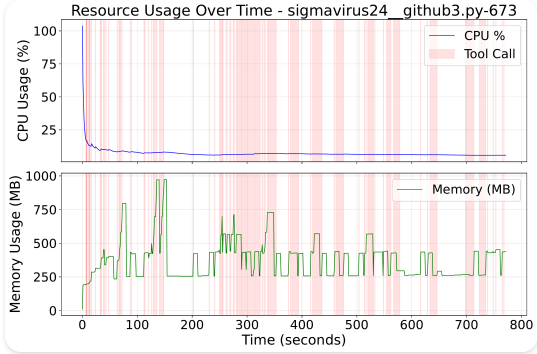


- **Memory, not CPU**, is the primary bottleneck (CPU avg <13%, memory peaks 2–4 GB)
- Resource consumption: **a ~185 MB framework baseline plus tool-call bursts**

Resource Time Series

Q: What drives resource bursts?

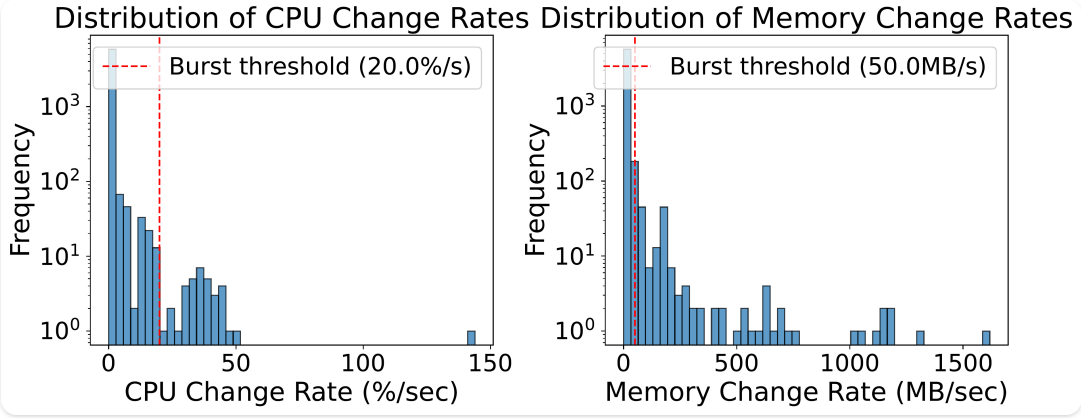




- Resource consumption is determined by what the tool *does* (e.g., pytest vs. git status), not which tool is invoked
  - **Bash calls differ by 13.7× in peak memory**
- **98.5% of memory bursts** occur during **tool calls**

Burst Dynamics & Variability

Q: How fast and how variable are resource changes?



- Resource bursts last **1–2 seconds**
- Peak-to-average ratio up to **15.4×**
- CPU-memory correlation varies by task **(−0.84 to +0.50); co-directional change cannot be assumed**

Characterization Summary

1. **OS execution dominates**: 56–74% of latency is tool execution + initialization, not LLM
2. **Memory is the bottleneck**: CPU avg <13%, but memory peaks 2–4 GB
3. **Demands are unpredictable**: 20× across tasks, 1.8× across runs; CPU/memory are decoupled

- **As LLM latency drops, OS overhead matters even more**
- Resource management for AI agents must operate at **tool-call granularity**

Agent vs. Cloud Workloads

Q: How different from existing cloud workloads?

- **Longer-lived than serverless**: tasks last **5–11 minutes**, not **100 ms to 2 s**
- **Far more memory-bursty than cloud baselines**: up to **15.4× peak/avg**, vs. roughly **1.5×–3×** in prior cloud studies
- **Less CPU-bound than batch, but memory-constrained**: CPU averages **<13%**, while memory peaks reach **2–4 GB**
- **Stateful and expensive to kill/restart**: the same task varies **1.8× across runs**, and killing loses context/progress

Three Mismatches

Q: Why do existing resource management tools fail?

	Static Limits	**Reactive Control**	**Predictive Scaling**
Representative tools	cgroup limits; K8s QoS	PSI + oomd	VPA; Autopilot
Assumes	Known peak; stable demand	Gradual pressure; kill OK	Repeatable; history valid
Agent	15.4× peak/avg; tool-call-level demand	1–2s burst; kill loses context	1.8× variance; history unstable
Mismatch	**Granularity**	**Responsiveness**	**Adaptability**

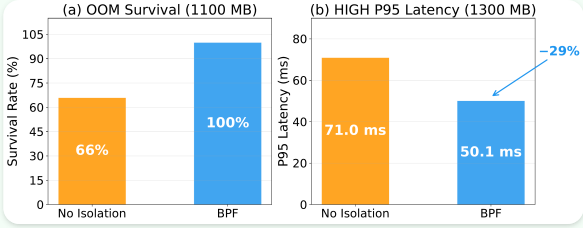
- **Granularity**: Too coarse-grained; `memory.max` at peak wastes >90%, while at average it causes OOM
- **Responsiveness**: Too slow; 3 GB/s bursts are too fast for user-space tools (tens of ms reaction time)

AgentCgroup Design and Preliminary Eval

Design

- Tool-call-aligned child cgroups: separate framework baseline from tool bursts
- In-kernel eBPF enforcement: `sched_ext` for CPU, `memcg_bpf_ops` for memory
- Intent-driven adaptation: env vars upward, stderr feedback downward

Preliminary Evaluation



- **Tight memory (1100 MB for ~1233 MB demand)**: survival improves from **66% → 100%**
- **Moderate memory (1300 MB)**: HIGH-priority P95 allocation latency drops **29%** (**70.97 → 50.14 ms**)
- Overhead is negligible: **P50 +0.3%**, total completion time **-1.1%**

Limitations & Future Work

Limitations

- Requires patched kernel for prototype (`memcg_bpf_ops` under review)
- Preliminary evaluation uses **3 replayed traces**, not live end-to-end agents
- Characterization covers **one agent framework** on **one benchmark**

Future Work

- Live multi-tenant evaluation at larger scale
- Agent-aware adaptation across more frameworks and tasks
- Extend control beyond CPU/memory to startup and orchestration overhead

Thank You

Questions?

References

AgentCgroup Paper (arXiv:2602.09345) · AgentCgroup GitHub